2025 1st International Conference on AIML-Applications for Engineering & Technology (ICAET)
Pune, India. Jan 16-17, 2025

A Comparative Survey of Text Summarization For Indian Languages

Bhairavi Bhuyar
B. Tech Computer Engineering Student
MKSSS's Cummins College of Engineering for Women,
Pune, India
bhairavi.bhuyar@cumminscollege.in

Sakshi Mandke
Assistant Professor
MKSSS's Cummins College of Engineering for Women,
Pune, India
sakshi.mandke@cumminscollege.in

2025 1st International Conference on AIML-Applications for Engineering & Technology (ICAET) | 979-8-3503-5561-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICAET63349.2025.10932170

***Abstract*—In Natural Language Processing (NLP), text summarization is an essential activity that aims to reduce massive text volumes into shorter, coherent versions while maintaining important information. While summarizing technologies have advanced greatly for major languages such as English, the development and implementation of summarization for Indian languages is still in its infancy. This research includes a comprehensive comparative review of text summarization methodologies in six major Indian languages, as well as a uniform evaluation framework. By methodically examining both extractive and abstractive summarizing techniques, we pinpoint important research voids and offer cross-linguistic perspectives that can direct further investigation. Through this analysis, we uncover previously unidentified relationships between language characteristics and summarization approach effectiveness, giving a structured roadmap for future improvements in Indian language text summarization.**

Keywords—Text Summarization, Extractive Summarization, Abstractive Summarization, Language Models, Evaluation Metrics

I. Introduction

Text summarization uses algorithms to condense enormous amounts of text into concise summaries that emphasize vital information. The basic goal of text summarizing is to determine the key concepts of the text and its sub-themes and generate a compact matter [1]. Summarization can simplify legal papers for lawyers, generate abstracts for researchers, and give brief summaries of customer interactions to improve service quality.

Artificial intelligence algorithms designed for different types of information are developed and utilized to do this. The world of papers that contain text is vast and growing every day on the World Wide Web [1]. The majority of data on the Internet exists as unstructured natural language text, making it difficult to extract meaningful information as compared to more structured databases. Summarization saves storage and time, enabling users to swiftly and easily comprehend the main points of the original sources without having to go through each one individually [2]. Figure 1 illustrates one such example of summary generation for the English language.

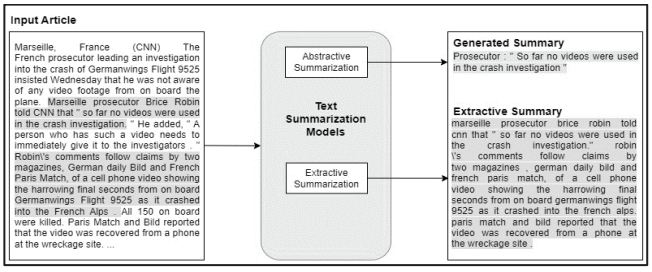

Fig. 1. Example of Text Summarization

Summarization in English is well established, but multilingual and regional languages, particularly Indian languages, have gotten little attention [3]. This research assesses the performance of summarization algorithms for a variety of Indian languages.

II. Approaches To Text Summarization

There are two different types of text summarizing approaches: abstractive, and extractive. Abstractive summarization resembles human comprehension, requiring information beyond the text, although it is more resource-intensive and less accurate than human summaries. As a result, extractive approaches, which generate summaries by choosing keywords or phrases from the source text, have grown in popularity. Figure 2 shows text summarization classified by output type. Extractive approaches identify significant sections of the text, whereas abstractive techniques rewrite the information using new terminology. Comparing various strategies proves difficult [4]. This study compares multiple abstractive as well as extractive summarization methods, including language models for Indian regional languages.

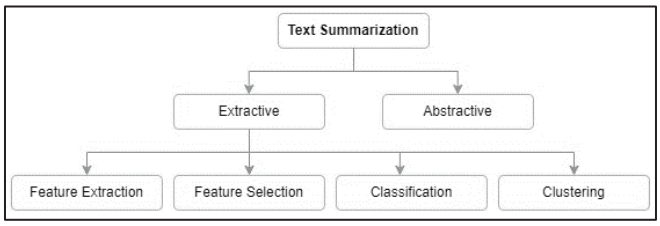

Fig. 2. Classification of Text Summarization

Extractive summarization attempts to extract objects from a set without modifying them. Extractive summarizers accept sentences as input and return a probability vector. The entries in this vector show the likelihood of the sentence appearing in the summary [5]. The most appropriate sentences are picked on the basis of summary's length. Hence, extractive text summarizing uses the text's raw structures, phrases, or sentences to generate a summation that solely uses content from the source material [6].

Efforts have been made to understand and explain a document's main concepts in clear natural language [7]. Abstractive summarizers use linguistic approaches to evaluate and understand text which is more difficult since they require rephrasing the content to create a summary using new words that were not there in the original text [8]. Despite its complexity, abstractive summarization outperforms extractive approaches by closely resembling human-generated summaries, making it more meaningful [9].

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.

An effective summary for both types should maintain the logical flow of essential concepts and ideas from the original text, reduce repetition, provide clarity and coherence, and preserve meaning, even in long phrases. Furthermore, the final summary should be brief while expressing the main points from the source. This can be a difficult endeavour due to the nonlinear structure of Indian languages and the scarcity of linguistic materials. A dearth of relevant information in Indian languages exacerbates the problem of making an algorithm to grasp regional languages. In contrast to the English language, no libraries like Gensim, NLTK, TextBlob, etc. are available [10]. Due to the non-identical scripts, the phonetics of the Indic languages are considerably different. Text summarization can be accomplished effectively by employing various approaches that address the challenges posed by regional languages. Some of which are discussed in greater depth in this paper.

The early approaches to summarization focused on extractive methods, relying on statistical techniques such as word frequency, TF-IDF, and ranking algorithms like TextRank or LexRank to determine and retrieve the pivotal sentences. While effective, these strategies frequently produced disconnected or incoherent summaries. Following sections enlist and explains the various algorithms used in summarization.

A. Algorithms for generating Abstractive Text Summary

Recent technological advances have resulted in the development of abstractive summarization methods that have dramatically improved the quality of generated summaries by providing coherent and contextually relevant text. These techniques can generally be categorized into four major categories: Graph-based approaches that use graph structures to represent relationships between sentences or words; Tree-based methods that portray text as hierarchical trees to effectively express dependencies; Rule-Based methods use predetermined rules or templates based on linguistic insights, while Deep Learning methods use neural networks such as transformers and Seq2Seq models to construct context-aware and semantically rich summaries. Figure 3 shows the classification of abstractive text summarization approaches.

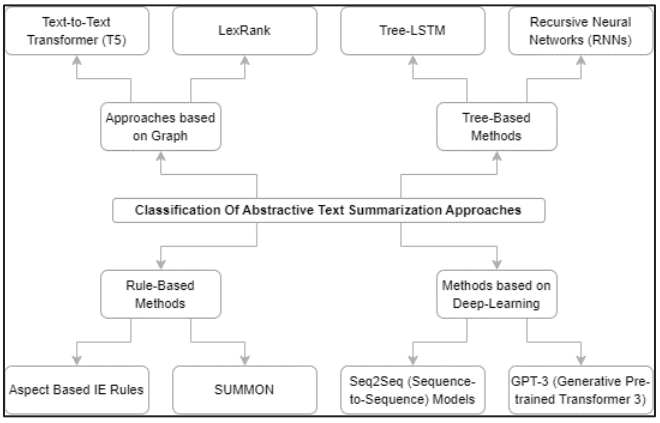

Fig. 3. Classification of Abstractive Text Summarization Approaches

1) Stacked LSTM based and Attention-based Sequence To Sequence (Seq2Seq) Neural Network

A Seq2Seq model transforms one sequence of items (such as letters, words, or time series) into another. Neural Machine Translation, for example, uses the Seq2Seq framework to convert input data from one language into output data from another. A Seq2Seq model has two primary parts: an encoder and a decoder. LSTM is employed in both because it captures long-term dependencies and avoids the issue of vanishing gradient [10]. In the Time Distributed Stacked LSTM Model, the encoder has two LSTMs, both of which have True return sequences and states. The Attention-based LSTM Model, on the other hand, has three stacked LSTMs with True return states and sequences, as well as a concatenated Attention Layer for enhanced performance.

2) Sequence-to-sequence Bi-directional RNNs

The model employs bidirectional RNNs composed of forward as well as backward recurrent neural networks, with LSTMs and an attention-based technique in the encoding and the decoding layer. It produces summaries consecutively. The encoder turns input phrases into a vector of fixed-length that the decoder uses to create an output set [11]. Both RNN layers are trained simultaneously to maximize the target text sequence's conditional probability [11]. Hidden units are used to boost memory and training efficiency.

3) Encoder/decoder, transformer-based model

The encoder-decoder architecture converts an input sequence to a vector of fixed-length, which is then decoded to provide an output sequence, eliminating the need for a separate feature extraction stage. The encoder turns input text into token IDs, which are sent through an embedding layer to build dense vector representations before being returned by an LSTM layer. The decoder gets the decoded input and summary token IDs before embedding and processing the summary through another LSTM layer. It generates a context vector using an attention method, which is then coupled with the current hidden state and fed through a dense layer to anticipate the next syllable in the summary.

4) Aspect-based IE rules

In abstractive text summarization, these criteria guide aspect-based information extraction (IE), which aids in the extraction and preservation of significant elements while providing short summaries. The IE rules consider language complexity and function as a filter for huge texts. They use a variety of synonymous verb and noun forms to define roles or characteristics, extracting crucial data using techniques such as pattern matching. Aspect-based IE interprets unstructured text to help with tasks such as opinion mining, product review analysis, and information retrieval.

B. Algorithms for generating Extractive Text Summaries

Several algorithms have been developed for extractive summarizing, with criteria such as summary length, text domain, and processing capacity determining method selection. This section describes the many algorithms used for this purpose. Figure 4 shows classification of approaches for extractive text summarization.

1) K-Means Clustering

A commonly used unsupervised learning strategy that groups data points based on underlying patterns. The method chooses K centroid places and allocates each data point to the closest centroid. As the process iterates, the centroids move to better represent the data. The number of clusters created is determined by the input text's size. Too few clusters may misrepresent the meaning of the text, while too many can result in over-extraction, which defeats the aim of summary.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.

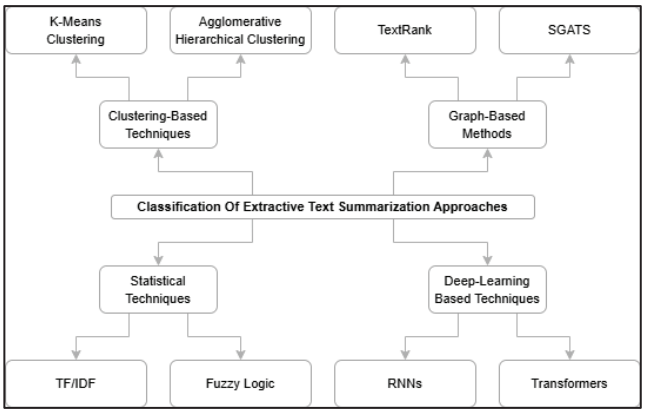

Fig. 4. Classification of Extractive Text Summarization Approaches

2) SGATS (Semantic Graph-based approach for Automatic Text Summarization)

The method creates a semantic network from the original text by constructing relationships between phrases using prior information. Once the semantic network has been created, various theoretical metrics are used to classify the document sentences based on their semantic scores [12]. It converts an input document (D) into a modified document (D') by deleting all nouns from sentences. It then creates a semantic network where each sentence serves as a node and the edges are formed using semantic linkages. A square matrix depicts sentence relationships, with entries of the total of semantic weights between noun terms in sentence pairs. Weights are determined by the semantic relationship between nouns in different phrases. The finished summary consists of the highest-ranked sentences, the number of which changes based on the summary length and other criteria.

3) RNN (Recurrent Neural Network)

The RNN, a kind of neural network that is designed to process sequential input (such as word sequences), is a popular solution for many NLP jobs [13]. RNNs process sequential input while keeping their internal state, recording dependencies between sequence pieces. Their connections form directed cycles, allowing for dynamic temporal activity, making them appropriate for sequence-based tasks. When trained on text and accompanying summaries, RNNs can recognize significant sentences and generate extractive summaries. Encoder and decoder components frequently employ RNN versions such as GRU or LSTM. This is due to their ability to capture long-term interdependence while avoiding the challenge of vanishing gradients [13].

4) Text-rank algorithm

The graph-based ranking system assigns value to nodes i.e. the vertices in a graph, with a link between the two vertices serving as a suggestion. The number of votes a vertex receives indicates its relevance. To use the TextRank approach, a graph is created from the text to rank the units. In sentence extraction, each phrase is represented by a vertex, and sentence similarity is calculated using common tokens or syntactic filters. The produced graph is strongly linked, with each edge assigned a weight depicting the connections made between distinct sentence pairs in the content [14].

5) TF/IDF (Term Frequency-Inverse Document Frequency)

The TF-IDF is an analytical method for determining the relevance of a word on the basis of its frequency in the document (TF) and overall significance (IDF). The resulting score represents a word's importance in both the document and the collection. Higher-scoring words are regarded as more important. Sentence scores are subsequently calculated by adding the TF-IDF word scores, with sentences containing skeleton or cue words being given extra weight [15].

6) Fuzzy Logic

The Fuzzy Logic model creates directives with unclear property weights to create an appropriate outline [16]. Unlike binary logic, which only deals with true or false values, fuzzy logic accepts different degrees of truth, making it excellent for summary tasks where sentence relevance is subjective and context-dependent. Using triangle membership functions, the fuzzifier classifies text as 'less, ‘medium,' or 'greater.' The inference engine uses IF-THEN rules to determine sentence relevance based on these values. The fuzzy rule-based method assigns relevance ratings, which are then converted into exact values via defuzzification, allowing for practical sentence inclusion decisions based on significance.

C. Language Models

Language models are essential in NLP for efficiently understanding, producing, and altering human language. Traditional NLP systems used less flexible statistical and rule-based methodologies. However, recent improvements in deep learning have resulted in transformer models, which provide more capabilities and flexibility. Pretrained models, such as BERT and GPT, allow task-specific fine-tuning, whilst cross-lingual models aid in multilingual comprehension and summarization across languages.

1) BERT (Bidirectional Encoder Representations from Transformers)

BERT is a transformer-based model pre-trained using a large multilingual dataset via self-supervised learning. It was trained on unlabelled texts, utilizing widely available data to create labels and inputs autonomously. To be more precise, it was pre-trained with the Masked language Modelling and Next Sentence Prediction objectives [17]. BERT was originally created for categorization and question answering but has now been repurposed for summarization. Its architecture is well-suited for detecting crucial sentences or phrases because of its ability to absorb linguistic nuances. BERTSUM, a variation tailored for summarizing, exemplifies BERT's adaptability in a variety of NLP scenarios [18].

2) BART (Bidirectional and Auto-Regressive Transformer)

BART, which combines the advantages of bidirectional and auto-regressive transformers, is a highly efficient text generating tool for summarizing tasks [18]. BART uses a conventional left-to-right decoder and a bidirectional encoder. It understands and generates well-organized summaries with ease. BART is good at comprehension tasks, but it shines when it comes to text production.

3) T5

T5, an encoder-decoder model, was trained on both supervised and unsupervised tasks before being modified to text-to-text format. Its architecture translates every activity into a simple format using text sequences as input and output, making it incredibly adaptable. T5's ability to provide concise and contextually rich summaries is enhanced by pre-training on a range of tasks [19].

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.

D. Evaluation Metrics

Evaluation criteria are critical for determining summaries' efficacy and quality. They guarantee that summarization results are correct, cohesive, relevant, and true to the source content. Both extractive and abstractive summarization require proper evaluation to lead changes and compare models. Machine metrics assess precision, recall, and accuracy by checking the system-generated summary with a reference summary. Recall measures the amount of relevant information recorded, whereas accuracy measures the absence of irrelevant information. The F1-score strikes a balance between these two characteristics [20].

Several common evaluation metrics are used in summarization:

1) BLEU (Bilingual Evaluation Understudy):

It evaluates the accuracy of n-grams by comparing the generated and referenced summaries. It is simple and effective, although it emphasizes clarity and may punish correct summaries with varied wording.

2) ROUGE-N:

It calculates the match between words of the generated summary, bigrams, or sequences and the reference summary. ROUGE-N evaluates n-gram matches, whereas ROUGE-L examines the longest mutual sequence segment, providing insights into fluency and coherence. However, it just evaluates overlap and may not fully reflect overall coherence or consistency.

3) METEOR (Metric for Evaluation of Translation with Explicit ORdering):

It advances on BLEU by taking into account stemming, synonyms, and word order, making it beneficial for summarization as it allows for more flexible evaluation depending on meaning. This measure is better to capture meaning but computationally more expensive.

4) BERTScore:

It evaluates summaries based on semantic similarity rather than word overlap, using contextual embeddings from models such as BERT. So, while it is preferable to capture deeper meaning and context, doing so can be computationally intensive.

These parameters contribute to high-quality and concise summaries. While summary measures for major Indian languages are comparable to those for English and other languages, the diversity of Indian languages, complicated grammatical structures, and limited resources present distinct obstacles. Despite these limitations, essential assessment requirements are consistent. Efforts are being made to adjust existing measures and create new tools to overcome language difficulty. Multilingual evaluation with multilingual embeddings and pre-trained models is an emerging technique. Furthermore, human-generated summaries will be useful, with native speakers evaluating them on informativeness, coherence, relevance, and fluency.

III. Application of the Algorithms to Indian Regional Languages

A. Hindi

1) Abstractive

a) Stacked LSTM based and Attention-based Sequence To Sequence (Seq2Seq) Neural Network:

Karmakar et al. [10] demonstrated that the Stacked LSTM model demonstrated impressive accuracy, scoring 0.583 after a brief training period. While, the Attention LSTM model, despite the longer training period, showed convergence with significantly fewer training and test losses, resulting in a higher score of 0.638.

2) Extractive

a) K-Means Clustering:

Kumari et al. [3] evaluated the model using four ROUGE F1-scores on both validation and testing sets. For validation, the model received ROUGE-1 F1-scores of 0.2985, 0.1429, 0.1118, and 0.0999. On the testing sets, the model recorded F1-scores for ROUGE-1 of 0.3272, ROUGE-2 of 0.1739, ROUGE-3 of 0.1395, and ROUGE-4 of 0.1257.

b) SGATS (Semantic Graph-based approach for Automatic Text Summarization):

Joshi et al. [12] illustrated that the SGATS model outperformed TextRank. Within the SGATS paradigm, graph theory metrics like Out-Degree, Eccentricity, and Weighted Out-Degree yielded the most effective summary sentences. However, one shortcoming of the SGATS model is that it only summarizes nouns, ignoring phrases from other parts of speech.

3) Language Models

a) BART:

Deshpande et al. [18] reported that the model achieves a ROUGE-1 score of 58.38, ROUGE-2 score of 48.31, and ROUGE-L score of 44.25.

b) T5:

Deshpande et al. [18] demonstrated that the model achieves a ROUGE-1 score of 60.72, ROUGE-2 score of 51.02, and ROUGE-L score of 47.11.

B. Marathi

1) Abstractive

a) Stacked LSTM based and Attention-based Sequence To Sequence (Seq2Seq) Neural Network:

Karmakar et al. [10] demonstrated that the Stacked LSTM model produced a score of 0.583 after a brief training period, whereas the Attention LSTM model comparatively achieved a higher score of 0.638.

2) Extractive

a) RNN:

Chaudhari et al. [13] reported that the model's application to Marathi produced an average precision of 0.319 and an overall recall of 0.342. The overall accuracy was 0.323. However, it was observed that the precision could potentially be enhanced by employing an alternative model.

b) Text-Rank:

Rathod [14] illustrated that the Text-Rank model received an average ROUGE-2 score of 0.7059, showing its ability to generate logical summaries from Marathi texts. Furthermore, results indicate that similarity-based summarizing strategies are more efficient and accurate than position-based methods.

3) Language Models

a) BART:

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.

Deshpande et al. [20] stated that the model shows a strong performance with a ROUGE-1 score of 27.15.

C. Bengali

1) Abstractive

a) Bi-directional RNNs, Attention model, Encoding, Decoding:

Talukder et al. [11] demonstrated that by using an attention-based encoder with hyperparameters such as 70 epochs, an RNN size of 256, a learning rate of 0.001, a batch size of 2, a keep probability of 0.75, and the Adam Optimizer, the model obtained maximum accuracy in predicting summaries. However, it struggled to generate summaries from long texts, resulting in summaries with a low word count.

2) Extractive

a) TF/IDF, K-Means Clustering:

Akter et al. [15] illustrated that the two models generated summaries with 30% of the original document's sentences, demonstrating their capacity to reduce text while retaining its substance. The technique's linear time complexity of O(n) indicates efficiency and scalability with larger datasets. However, the summary phrases occasionally lacked sequence synchronization, hurting the summaries' consistency and flow.

3) Language Models

a) BART:

Deshpande et al. [20] reported that the model achieves a ROUGE-1 score of 19.77.

D. Gujarati

1) Extractive

a) K-Means Clustering:

Kumari et al. [3] demonstrated that the model for Gujarati was evaluated using four ROUGE F1-scores on both validation and testing sets. For validation, the model received ROUGE-1 F1-scores of 0.2985, 0.1429, 0.1118, and 0.0999. On the testing sets, the model recorded F1-scores for ROUGE-1 of 0.3272, ROUGE-2 of 0.1739, ROUGE-3 of 0.1395, and ROUGE-4 of 0.1257.

2) Language Models

a) BART:

Deshpande et al. [18] demonstrated that the model achieves a ROUGE-1 score of 26.11, ROUGE-2 score of 16.51, and ROUGE-L score of 12.41.

b) T5:

Deshpande et al. [18] also illustrated that the model achieves a ROUGE-1 score of 26.07, ROUGE-2 score of 16.60, and ROUGE-L score of 12.58.

E. Urdu

1) Abstractive

a) Encoder/decoder, transformer-based model:

Raza et al. [4] demonstrated that the model scored high on ROUGE, suggesting its usefulness in summarizing tasks, with mean F1-scores of 0.43 for ROUGE-1, 0.25 for ROUGE-2, and 0.23 for ROUGE-L. It outperformed standard approaches such as SVM's and logistic regression, and even matched the quality of human-generated summaries, indicating its potential for improving Urdu language summarization techniques.

2) Extractive

a) Fuzzy Logic:

According to Ali et al. [16], the Fuzzy Logic model applied to Urdu extractive text summarization improved flexibility and performance, hence improving the sequence-to-sequence network. It outperformed human-generated summaries in terms of recall but struggled with duplication, resulting in worse precision.

3) Language Models

a) BERT:

Munaf et al. [19] evaluated the model on the BBC Urdu dataset, with ROUGE-1 at 48.75, ROUGE-2 at 32.45, ROUGE-L at 42.63, and a BERTScore of 77.83. The DW Urdu dataset scored well, with ROUGE-1 at 34.23, ROUGE-2 at 12.22, ROUGE-L at 23.85, and BERTScore at 72.64.

F. Kannada

1) Abstractive

a) Aspect-based IE rules:

According to Shilpa G V and Shashi Kumar D R [7], the model preserved crucial information and attained good language quality, as evidenced by the Mean Opinion Score. It also outperformed extractive approaches in terms of compression ratios and accuracy, ranging from 70 to 80 percent. However, the IE rule repository may require expansion in order to adequately address the language's various syntactic features.

2) Language Models

a) BART:

Deshpande et al. [20] reported that the model achieves a ROUGE-1 score of 33.80.

IV. Comparisons

Table 1 outlines the techniques and language models utilized for the different languages. (✓) represents the mapping of approach types, language models, and evaluation metrics to languages. Table 2 contains information on numerous terms used in the previous table.

TABLE I. Comparisons Table

Language	Approach Type										Language Model			Evaluation Metric	
	Abstractive				*Extractive*										
	A 1	*A 2*	*A 3*	*A 4*	*E 1*	*E 2*	*E 3*	*E 4*	*E 5*	*E 6*	*L 1*	*L 2*	*L 3*	*M 1*	*M 2*
Hindi	✓				✓	✓						✓	✓	✓	
Marathi	✓						✓	✓				✓		✓	
Bengali		✓			✓				✓			✓			
Gujarati					✓							✓	✓	✓	
Urdu			✓							✓	✓			✓	✓
Kannada				✓								✓			

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II. LIST OF ABBREVIATIONS USED

Approach Type	
Abstractive	
A1	Stacked LSTM based and Attention-based Sequence To Sequence (Seq2Seq) Neural Network
A2	Sequence-to-sequence Bi-directional RNNs
A3	Encoder/decoder, transformer-based model
A4	Aspect-based IE rules
Extractive	
E1	K-Means Clustering
E2	SGATS (Semantic Graph-based approach for Automatic Text Summarization)
E3	RNN (Recurrent Neural Network)
E4	Text-rank algorithm
E5	TF/IDF (Term Frequency-Inverse Document Frequency)
E6	Fuzzy Logic
Language Models	
L1	BERT
L2	BART
L3	T5
Evaluation Metrics	
M1	ROUGH-N
M2	BERTScore

V. CONCLUSION

This survey has explored various approaches to text summarization for Indian regional languages, comparing abstractive, extractive, and language model-based techniques across Hindi, Marathi, Bengali, Gujarati, Urdu, and Kannada. The study showcases a wide range of methodologies, from traditional extractive procedures to advanced neural models, while also revealing the unique constraints faced by linguistic structures, scripts, and limited NLP resources. The effectiveness of different methods vary across languages, with LSTM-based approaches showing promise for Hindi, while BERT-based models performing well for Urdu. This paper underlines the critical necessity for larger datasets and pre-trained models for Indian languages, addressing a major issue in data-intensive techniques. This comparative study sheds light on resource limitations and potential improvements in text summarizing systems for underrepresented languages.

Future research should concentrate on creating larger datasets for Indian languages, cross-lingual transfer learning, and hybrid systems that have the strengths of extractive as well as abstractive approaches. Improving evaluation metrics to better capture the linguistic intricacies of Indian languages, as well as resolving specific difficulties such as dialectal variances, will be crucial. As NLP technologies continue to advance, text summarization for Indian languages presents a rich area for innovation, with potential applications in education, media, and information access for millions of users. Continued study in this field has the potential to improve information availability and processing for a vast and linguistically diverse population.

REFERENCES

[1] Dhanya P.M, Jathavedan M, "Comparative Study of Text Summarization in Indian Languages", International Journal of Computer Applications, Volume 75 No.6, pp. 17

[2] Vishal Gupta, "A Survey of Text Summarizers for Indian Languages and Comparison of their Performance", Journal of Emerging Technologies in Web Intelligence, Vol. 5 No. 4, pp. 361

[3] Kirti Kumari, Ranjana Kumari, "An Extractive Approach for Automated Summarization of Indian Languages using Clustering Techniques", CEUR Workshop Proceedings, Vol. 3395

[4] Ali Raza, Hadia Sultan Raja, Usman Maratib, "Abstractive Summary Generation for the Urdu Language", arXiv:2305.16195 [cs.CL], Vol. 1

[5] Aakash Sinha, Abhishek Yadav, Akshay Gahlot, "Extractive Text Summarization using Neural Networks", arXiv:1802.10137 [cs.CL], Vol. 1, pp. 1

[6] Derek Miller, "Leveraging BERT for Extractive Text Summarization on Lectures", arXiv:1906.04165 [cs.CL], Vol.1, pp. 1

[7] Shilpa G V, Shashi Kumar D R, "Abs-Sum-Kan: An Abstractive Text Summarization Technique for an Indian Regional Language by Induction of Tagging Rules", International Journal of Recent Technology and Engineering (IJRTE), , Volume-8, Issue-2S3, pp. 1028

[8] Ayesha Ayub Syed, Ford Lumban Gaol, Tokuro Matsuo, "A Survey of the State-of-the-Art Models in Neural Abstractive Text Summarization", IEEE Access, Vol. 9, pp. 1

[9] C. Sunitha, A. Jaya, and A. Ganesh, "A study on abstractive summarization techniques in Indian languages," Procedia Computer Science, vol. 87, pp. 25–31, 2016.

[10] Rishabh Karmakar, Ketki Nirantar, Prathamesh Kurunkar, Pooja Hiremath, Prof. Deptii Chaudhari, "Indian Regional Language Abstractive Text Summarization using Attention-based LSTM Neural Network", 2021 International Conference on Intelligent Technologies (CONIT), DOI:10.1109/CONIT51480.2021.9498309, pp. 2

[11] Md Ashraful Islam Talukder, Sheikh Abujar, Abu Kaisar Mohammad Masum, Fahad Faisal, Syed Akhter Hossain, "Bengali abstractive text summarization using sequence to sequence RNNs", 10th ICCCNT 2019, IEEE - 45670, pp. 1

[12] Manju Lata Joshi, Nisheeth Joshi, Namita Mittal, "SGATS: Semantic Graph-based Automatic Text Summarization from Hindi Text Documents", ACM Trans. Asian Low-Resour. Lang. Inf. Process. 20, 6, Article 102 (September 2021), 32 pages

[13] Anishka Chaudhari, Akash Dole, Deepali Kadam, "Marathi text summarization using neural networks", International Journal of Advance Research and Development, Volume 4, Issue 11, pp. 1

[14] Yogeshwari V. Rathod, "Extractive Text Summarization of Marathi News Articles", International Research Journal of Engineering and Technology (IRJET), Volume: 05, Issue: 07, pp. 1208

[15] Sumya Akter, Aysa Siddika Asa, Md. Palash Uddin, Md. Delowar Hossain, Shikhor Kumer Roy, Masud Ibn Afjal, "An Extractive Text Summarization Technique for Bengali Document(s) using K-means Clustering Algorithm", IEEE 2017, pp. 1

[16] Zeshan Ali, Jawad Ali, Zhang lining, Zeeshan, Niaz Muhammad, "Automatic Text Summarization for Urdu Roman Language by Using Fuzzy Logic", Journal of Autonomous Intelligence (2020), Volume 3, Issue 2, pp. 23

[17] Vakada Lakshmi Sireesha, "Text Summarization for Resource-Poor Languages: Datasets and Models for Multiple Indian Languages", pp. 3

[18] A Deshpande, P. ., & Jahirabadkar, S. . (2024). Comparative Analysis of Lexical Chain, Bidirectional Encoder Representations from Transformers (BERT) and Graph based Approaches for Condensation of Low Resource Language Documents. International Journal of Intelligent Systems and Applications in Engineering, 12(16s), 08–15. Retrieved from https://ijisae.org/index.php/IJISAE/article/view/4777

[19] Mubashir Munaf, Dr Hammad Afzal, Naima Iltaf, Khawir Mahmood, "Low resource summarization using pre-trained language models", arXiv:2310.02790v1 [cs.CL]

[20] Deshpande, P., Jahirabadkar, S. (2022). A Survey on Statistical Approaches for Abstractive Summarization of Low Resource Language Documents. In: Zhang, YD., Senjyu, T., So-In, C., Joshi, A. (eds) Smart Trends in Computing and Communications. Lecture Notes in Networks and Systems, vol 286. Springer, Singapore. https://doi.org/10.1007/978-981-16-4016-2_69

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:06:18 UTC from IEEE Xplore. Restrictions apply.